SmartBrew AI - Group 28 - Lab 3 AI appendix

Updated 2026-10-21. Tool: Codex desktop application. Tasks A (elicitation), B (complete code generation/testing) and C (documentation/repository): GPT-6.1 Sol, the exact selector label confirmed by the student. Exact backend build: not exposed. Earlier Lab 1/2 model versions: unknown, not reconstructed.

Record scope and prompt strategy

This appendix includes the main supplied request, key responses and clarifications. Markdown whitespace is normalized. The later Lab 1/course-policy context is explicitly summarized rather than represented as a full verbatim export. Generated source files in the Git repository are the code responses. This record contains no private model reasoning, invented prompts or student-written personal reflection.

Use the actual system description, feedback and course slides as grounding; constrain all requirements to the six-column lecture template; retain the submitted baseline; check each source capability against stories/requirements/backlog; separate source mandates from proposed design assumptions; select ten coherent demonstrable feature groups; implement and test approval, rejection, ownership, retry, safety and offline paths. This was one evolving chat request with student clarifications, not an invented sequence of separate prompting experiments.

Policy and baseline provenance

Previously generated Lab 2 correction: AI-GENERATED REVIEW REFERENCE ONLY - NOT FOR LAB 1 OR LAB 2 SUBMISSION. The general course rules supplied after this reference was generated prohibit submitting generated text/models for those labs. Original submissions are unchanged. Lab 3 explicitly permits generation; disclosure does not override the Lab 1-2 restriction.

The preserved files in docs/baseline are exactly the submitted Lab 2 artifacts extracted from the supplied ZIP. This chat previously assisted creation of Lab 2; it would be inaccurate to label that submission an independently human-authored baseline. The comparison uses the actual submitted group version, then separates the feedback correction from the later LLM-assisted refinement. No later correction is silently substituted for the submitted baseline.

The submitted Lab 2 cannot honestly be called independently human-elicited; the comparison discloses that limitation for instructor review. The Lab 1 original is preserved as supplied. Generated Lab 2 corrections are review references only, not submission materials.

Main Lab 2 Correction / Lab 3 Prompt

Student message from 20 October 2026. The assignment text below is the input supplied by the student, rather than a claim that the assistant authored it. Source attachments: feedback `G28Lab2 (1).pdf`, submitted `Lab2_G28.zip`, and the supplied Lecture 2 and Lecture 3 PDFs.

G28Lab2 (1).pdf is the feedback we got. zip file contains our lab2 submission. according to feedback, and lab lectures 2 and 3, and if you need, lab2s guidelines: "Lab assignment 2:

This lab is about requirements and planning. For task A, we will continue to use the tool Astah. For task B, we recommend word or another textual tool with a PDF capability.

Task 2a:

Create a UML use case diagram for the system as described in the [System description for all lab assignments](https://canvas.mdu.se/courses/20908/pages/system-description-for-all-lab-assignments "System description for all lab assignments") , including the feature of your own design as you added during lab 1.

To be marked complete, the diagram must cover the functionality of the system and contain the following:

- At least 3 Actors
- At least 8 Use cases with:
- For each use case, a textual description (right click the use case and select "open use case description"). You may use [this template](https://canvas.mdu.se/courses/20908/files/3005789?wrap=1)[Download this template](https://canvas.mdu.se/courses/20908/files/3005789/download?download_frd=1) for describing the use cases. Import it into Astah via Tool -> Template Settings -> UseCase Description -> Import and then Apply.
- Correct relationships between actors, between use cases, and between actors and use cases.
- A clear indication of the feature you added yourself (if any), for example by adding a note to the diagram.

Task 2b:

In this assignment, you will write user stories and create backlogs for the development.

Follow the following steps to complete this task:

1. Read Chapters 2 (on product backlogs), 3 (on sprint planning), and 6 (on sprint backlogs) of "Scrum and XP From the Trenches".
2. Based on the system description and your added feature, write down a list of at least 10 **user stories** for the SmartBrew system. Your user stories need to cover the complete functionality of the system according to the description. A typical user story has the form *"As a <role> I want <goal/desire> so that <benefit>";* you can refer to the template from the slides.
3. (starting from the user stories) Define a set of **requirements** covering the key functionalities of the system (the requirements should be expressed using the template from the slides)
4. Create an initial **product backlog** with the requirements (the initial product backlog contains at least ID, name, importance, estimate, description, how to demo) using the template provided in the slides.". we need to fix, and then do lab3:

Lab Assignment 3: Software Engineering with LLMs

The goal of this assignment is to engage with core software engineering tasks using Large Language Models (LLMs) as virtual assistants. You will observe, guide, and critically evaluate the software development lifecycle when augmented by AI.

You are required to complete **three main tasks**:

1. **Task A:** LLM-Assisted Requirements Elicitation
2. **Task B:** Code Generation & Implementation
3. **Task C:** Documentation & Repository Management

Choice of LLM

You are free to use **any LLM or AI assistant of your choice** (e.g., ChatGPT/GPT-4, Claude, Gemini, GitHub Copilot, Llama, etc.). However, you must explicitly document which model(s) and version(s) you used for each task.

Task A: Requirements Elicitation & Critical Comparison

1. **Elicitation with LLMs:**

Conduct a requirements elicitation process for the System Description used across all Labs using your chosen LLM. Generate the following three artifacts (matching the required format from Assignment 2):
- Functional and Non-Functional Requirements
- User Stories
- Product Backlog
2. **Prompting & Transcript:**

You must document the prompt strategy you used to generate these artifacts. Include the main prompts and key responses as an appendix or separate document.
3. **Comparison with Assignment 2:**

Compare the LLM-generated artifacts with the human-elicited version you submitted for Assignment 2.
- Highlight differences clearly (e.g., via side-by-side text, Git diff format with red/green highlights, or detailed annotated lists).
- Detail what the LLM improved, missing aspects it failed to identify, or unneeded/incorrect requirements it suggested.
- If you reject LLM suggestions, you **must provide a short justification** for each rejection.

(Note for students who did not submit Assignment 2: You may either complete Assignment 2 first and then perform the comparison, or perform the LLM elicitation directly while fully documenting and justifying every step of your interaction with the LLM).

Task B: Code Generation & Execution

1. **Functionality Selection:** Select a core set of **10 features/functionalities (not trivial)** from the requirements elicited in Task A.
2. **Implementation:** Realize/code these 10 functionalities with LLM assistance.
3. **Executable Prototype:** Ensure a working user experience. The system must run smoothly and demonstrate the selected 10 functionalities clearly. It is not required a complete portability of the project (Can work on only one specific OS, like macOS, linux and windows).
4. In case of complete code generation, you need to add the conversation. If you used the LLM just as an assistant to revise the code it is not required to include the conversation.

Task C: Repository & Documentation

1. **Git Repository:** Host your codebase in a Git repository (e.g., GitHub).
2. **Code Documentation & Comments:**

All source code must be documented with clear comments. At a minimum, every function/method must include:
- Description of purpose
- Expected Input parameters
- Expected Output / Return values
3. **README Document:**

The repository must include a clear `README.md` containing:
- **System Overview:** Brief description of the project.
- **Environment & Prerequisites:** Compiler/runtime versions, libraries, and external dependencies required to run the code.
- **Execution Guide:** Step-by-step instructions on how to build, run, and test the 10 functionalities (whether via command-line arguments, scripts, CLI, or GUI navigation steps).

General Information & Constraints

- You may use any programming language, framework, or IDE. However, remember that the evaluation takes place on an independent machine. Your project **must include clear environment setup instructions** or configuration files (e.g., `requirements.txt`, `package.json`, `pom.xml`, or Dockerfile) so it can be built and run without version mismatch issues.

Key responses and student clarifications

Assistant:

> I'll read the feedback, submitted files, and lecture templates first. Then I'll use them to correct Lab 2 and build Lab 3's requirements, comparison, prototype, and documentation.

Assistant:

> I'm using the PDF skill to check the feedback annotations and lecture examples. I'll keep the original submission intact and create the revised work in your workspace so we can compare both versions honestly.

Assistant:

> The feedback gives Lab 2 a positive grade, but asks for fuller requirement descriptions, actor placement changes, and customization as the customer's starting point. The submitted PDF is missing the required preconditions and postconditions, so my earlier reassurance was too broad. I'll correct those and make Lab 3's comparison explicit about the AI assistance already used in Lab 2.

Assistant:

> The lecture template confirms six fields for each requirement: ID, Description, Inputs, Outputs, Precondition, and Postcondition. I'll retain your 13-item Lab 2 structure for the correction, then use a separate Lab 3 set to show additions such as saved recipes, transaction records, offline fallback, and waste safety.

Assistant clarification:

> For Lab 3's hosted Git repository, what GitHub repository URL should I use? I can prepare and test everything locally while you provide it.

Student:

> https://github.com/kaantanidir/Software_Engi_1_LAB3.git is the project github link

Assistant:

> For Lab 3, I'll build a local browser app around ten demonstrable features: accounts, menu browsing, customization, saved recipes, availability, recommendations, checkout/history, loyalty, receipts, and maintenance. Hardware inputs and external payments/email will be clearly labeled simulators; camera inference and a real local LLM will remain deferred.

Assistant:

> I'll use the GitHub repository you provided and check its contents before connecting the finished work. I also found two saved-model issues the earlier name checks missed: "Process Payment" contains the natural-language description, and "Analyze Mood" has no completed description. Both will be repaired.

Assistant clarification:

> What exact model name/version is shown in your app's model selector? Lab 3 requires that label in the model-use record.

Student:

> what AI model do I use? is that the question?

Assistant:

> Yes. Lab 3 asks you to name the AI model used for each task. I meant the model label shown for this chat, so I can record it accurately. We can mark the earlier Lab 2 model as unknown if you no longer know which one it was.

Student:

> GPT-6.1 Sol

Assistant:

> The prototype's main flows are in place. I'm now checking failure cases: declined payments must leave stock and points unchanged, checkout retries must not duplicate a purchase, and full waste bins must block ordering. The Astah correction also gives optional mood analysis and natural-language entry explicit extension points.

Assistant:

> The corrected Astah file has been saved and validated through Astah's API. All 13 use cases now have descriptions, with one mandatory ingredient-check relationship and three optional extensions. The first 16 business-rule tests also pass, including concurrent checkout retries, recipe ownership, offline payments, and waste/fault blocking.

Assistant:

> The browser walkthrough now passes the ordering, loyalty, receipt, maintenance, and offline-payment flows. I'm finishing the mobile checks and document layout review. I've also recorded the model label you confirmed, GPT-6.1 Sol, for Lab 3 Tasks A, B, and C.

Assistant:

> The final checks pass: 16 business-rule tests and 23 browser checks. I also fixed a retry bug found during review: offline pending orders now retain the customer's loyalty-point selection. The documentation includes that correction and the prototype's limits, so it doesn't imply real payment, camera, or LLM integration.

Code-generation outputs and observed revisions

The assistant generated the complete application and its tests from the request, not just a code review. The following files are the corresponding code responses retained in an executable form:

- `app.py`, `smartbrew/service.py`: HTTP adapter, accounts, ownership, recipes, recommendations, atomic checkout, receipts and maintenance.
- `web/index.html`, `web/styles.css`, `web/app.js`, `web/receipt.html`, `web/receipt.js`: usable browser interface.
- `tests/test_service.py`, `tools/browser_check.cjs`: failure-path and end-to-end verification.
- `tools/coursework_data.py`, `tools/build_documents.py`: elicited artifact records and document outputs.
- `tools/AstahRepair.java`: licensed-API model correction/validation.

Execution record, not invented conversational prompts:

1. Extracted actual feedback and baseline; rendered the lecture's image-only requirement/backlog templates.
2. Kept original source hashes and compared against the original PDF rather than the later correction.
3. Corrected empty/wrong Astah descriptions and relationships using the supported API. Native image export verified arrow directions and actor positions.
4. Implemented ten selected features and explicitly deferred hardware/real AI integrations. Considered/rejected cloud facial-image

processing, fabricated offline wallet approval, fake LLM/mood implementations and unrequested product features; justifications are in the report.
5. Initial unit-test execution was blocked by the desktop sandbox's temporary-directory restrictions. Test directories were moved inside the workspace; 16 business-rule tests passed.
6. Browser QA exposed asynchronous test-wait and relative-URL issues in the test harness, not verified application defects. The checks were changed to wait for the actual quote/status, use a base URL and execute DOM inspection in a self-contained function. The final recorded browser run passes all checks in `evidence/browser-results.json`.
7. A subsequent review found an actual application issue: pending-wallet retry originally reset the selected loyalty points. The assistant added persistent pending points, restored them in the payment form, and tested an offline request with 100 points through reconnect/retry. This illustrates why an initial passing run did not cover every interaction.
8. Documents and screenshots were rendered/reviewed, with final test and repository status recorded in `docs/QA_RESULTS.md`.

This record discloses that both the original baseline and new implementation involved AI. Students should add their own critical reflections or rejected suggestions if they subsequently change the generated work; no student-written evaluation is fabricated here.

Subsequent Lab 1 review and policy clarification - 2026-10-21

Student supplied `G28Lab1.pdf` as feedback, `G28Lab1.zip` as the submission, the Lab 1/system/general course instructions and the instructor's recommendation for additional experimentation. Main request excerpt (verbatim; the lengthy accompanying course quotations are summarized next):

> again pdf contains feedback, zip has submission.

> I have provided all lab 1,2,3 instructions, feedback of 1 and 2, submission of 1 and 2.

Source-context summary, not a verbatim prompt: general rules prohibit generated Lab 1-2 text/models and restrict AI to explanations/grammar; AI use needs a PDF appendix with tool/version/prompts. Lab 3 permits end-to-end generation. The instructor strongly recommends extra experimentation and requests a personally written reflection for any extra exercise performed.

Assistant response excerpt:

> The general instructions add an important distinction: Labs 1-2 prohibit submitting AI-generated text or models, while Lab 3 explicitly allows generation. I'll keep the generated Lab 2 correction labeled as a review aid and check that the repository makes that clear. The instructor's extra experiment also requires a reflection you write yourself.

Assistant question:

> For the instructor's optional experiment, which would you like to try yourself? I can prepare a repeatable exercise; you would write the personal reflection.

Student:

> do we need to, wouldnt disabling or mutating would decrease functionallityt of the project

Assistant:

> The message presents the extra experiment as recommended, rather than a required fourth task.
>
> A mutation would run only in a disposable test copy: deliberately break one rule, check whether a test catches it, then discard the copy. Your main project keeps all ten features. I'll finish the consistency review first; we can leave the optional experiment out for now.

Observed follow-up work: read-only supported-API inspection and native exports confirmed nine visible domain concepts, a positive Lab 1 grade with two component-related Partial findings, and no loyalty feature documented in Lab 1. Original hashes were preserved. A normal browser regression (HTTP 503 injected into an isolated QA browser) reproduced a blank menu caused by awaiting recommendations before state rendering. The UI was changed to render essential ordering first, time-limit optional ranking and retain standard-menu fallback. This is reliability validation, not an optional student experiment. The generated Lab 2 references were relabeled, a cross-lab review was added, and the required Lab 3 AI appendix was exported as PDF. No personally written student reflection was invented.